Building the Timber Man Game Step-by-Step using C++ and SFML

Dr. Mohammad Maksood Akhter
Assistant Professor

Department of Computer Science and Engineering

Feburary 25, 2026

Timber Man: Conceptual Overview

Timber Man is a deterministic real-time interactive system with stochastic branch generation.

- The player controls a lumberjack positioned either on the LEFT or RIGHT side of a tree.
- The objective is to maximize score by chopping the tree repeatedly.
- After each chop:
  - A new branch is generated at the top.
  - Existing branches shift downward.
- The player must switch sides strategically to avoid collision.
- A continuously decreasing timer introduces real-time pressure.
- The system combines:
  - Discrete events (keypresses),
  - Continuous time decay,
  - Random branch generation.

This makes the game an excellent model for studying state transitions and real-time simulation in C++.

Mathematical Abstraction of Game State

We define the complete game state at time t as:

$$S(t) = \big(P(t), B(t), T(t), Score(t), Alive(t)\big)$$

where:

- $P(t) \in \{LEFT, RIGHT\}$ Represents player position.
- $B(t) = \{B_0(t), B_1(t), ..., B_{N-1}(t)\}$ Branch positions array.
- $T(t) \in \mathbb{R}^+$ Remaining time.
- $Score(t) \in \mathbb{N}$ Player score.
- $Alive(t) \in \{0, 1\}$ Game status indicator.

Understanding Branch Transition and Collision Logic: I

The mathematical rules describe how the game state evolves after every chop action.

1. Branch Transition Rule

$$B_i(t+1) = B_{i-1}(t), \quad i = 1, ..., N-1$$

This means:

- Each branch shifts one level downward.
- The branch previously at position $i-1$ moves to position i.
- This creates the visual illusion of branches falling.
- Algorithmically, this is implemented using a reverse loop.

Understanding Branch Transition and Collision Logic: II

2. New Branch Generation

$$B_0(t+1) = Random(LEFT, RIGHT, NONE)$$

This means:

- A new branch is inserted at the top level.
- Its position is randomly selected.
- Randomness introduces uncertainty and difficulty.

Understanding Branch Transition and Collision Logic: III

3. Collision Condition

$$B_{N-1}(t) = P(t) \Rightarrow Alive(t+1) = 0$$

This means:

- The lowest branch is compared with the player position.
- If both are on the same side, collision occurs.
- The game transitions to Game Over state.

Interpretation: The entire falling mechanism is not physics-based; it is a discrete-time state shift in an array.

Timber Man: Game Interface

Initial game screen layout showing tree, player, and background.

Branch Shift Illustration

Branches shifting downward after each chop.

Before and After Chop

State transition from t to $t+1$.

Collision Condition Demonstration

Lowest branch aligns with player position resulting in Game Over.

Concepts We Will Learn

- Game Loop Architecture
- Event Handling
- Real-time Keyboard Input
- Sprite and Texture Management
- Enumerations in C++
- Arrays for State Simulation
- Collision Detection
- Delta Time and Frame Independence
- Basic Game Physics Simulation

Step 1: Creating the Game Window (Part 1)

```
#include <SFML/Graphics.hpp>
using namespace sf;

int main()
{
    RenderWindow window(VideoMode(1920, 1080),"Timber Man");
}
```

This is the minimal structure required to create an SFML application.

- Every SFML program begins with including the graphics module.
- The main function is the entry point of execution.
- The first object we create is the game window.

Without this window, no graphics can be rendered.

Step 1: Understanding SFML and RenderWindow (Part 2)

#include <SFML/Graphics.hpp>

- Imports the SFML Graphics module.
- Provides classes for:
  - Window management
  - Rendering
  - Sprites and textures
  - Text and shapes

RenderWindow

- Represents an operating-system-level window.
- Creates and manages the OpenGL rendering context.
- Acts as the main drawing surface.
- All game objects will be drawn onto this window.

The window is the core container of the entire game system.

Step 1: Resolution and Window Configuration (Part 3)

VideoMode(1920, 1080)

- Defines window width and height in pixels.
- Establishes coordinate boundaries:

$$x \in [0, 1920], \quad y \in [0, 1080]$$

- Determines the rendering viewport.

"Timber Man"

- Sets the window title.
- Helps identify the running application.

Important Note: The window object must persist during the entire execution. If it goes out of scope, the program terminates automatically.

Step 2: Basic Game Loop

The following structure implements the minimal real-time loop required to keep the window responsive and continuously render frames.

```
while(window.isOpen())
{
    Event event;
    while(window.pollEvent(event))
    {
        if(event.type == Event::Closed)
        window.close();
    }

    window.clear();
    window.display();
}
```

Game Loop Architecture (Part 1)

Every real-time interactive application follows a continuous execution model called the **Game Loop**.

The fundamental structure is:

$$Initialize \rightarrow Loop(Input \rightarrow Update \rightarrow Draw)$$

- **Initialize** — Load textures, create objects, set variables.
- **Input** — Capture user interaction (keyboard/mouse).
- **Update** — Modify game state based on input and time.
- **Draw** — Render updated state to the screen.

This cycle repeats continuously during gameplay.

Game Loop Architecture (Part 2)

Inside the program, the loop typically looks like:

$$while(window.isOpen())$$

1. **Handle Input**
   - Detect key presses
   - Process OS events
   - Capture user interaction
2. **Update Game State**
   - Move branches
   - Update timer
   - Check collisions
   - Modify score
3. **Render Frame**
   - Clear window
   - Draw sprites
   - Draw text
   - Display updated frame

Core Principle

The loop executes repeatedly until the window is closed.

Game Loop Architecture (Part 3)

The game loop transforms a static program into a dynamic system.

Mathematically, each iteration represents:

$$S(t+1) = f(S(t), Input(t), \Delta t)$$

where:

- $S(t)$ = current game state
- $Input(t)$ = user interaction
- Δt = elapsed time between frames

Why this works:

- The loop runs many times per second (e.g., 60 FPS).
- Each frame slightly modifies the state.
- Rapid updates create the illusion of motion.

Without the game loop, the program would execute once and terminate.

Texture: Formal Definition

A **Texture** in SFML is a memory-resident image resource loaded from a file and stored in GPU-accessible memory.

Definition:
A texture is a data container that stores pixel information used for graphical rendering.

- Loaded from image files (PNG, JPG, etc.).
- Contains raw pixel data.
- Stored in video memory (VRAM) when possible.
- Cannot be directly displayed on the screen.

Important Concept: A texture represents *data*, not a visual object.

Sprite: Formal Definition

A **Sprite** is a drawable graphical object that uses a texture to render an image on the screen.

Definition:
A sprite is a transformable entity that references a texture and can be positioned, rotated, scaled, and rendered.

- Acts as a visual representation of a texture.
- Has properties such as:
  - Position
  - Rotation
  - Scale
  - Origin
- Multiple sprites can share the same texture.

Important Concept: A sprite represents an *object in the game world*, while the texture represents the underlying image data.

Step 3: Loading Background (Part 1)

```
        Texture textureBackground;
        textureBackground.loadFromFile("graphics/background.png");
```

This step loads the background image into memory.

Texture textureBackground;

- Creates a texture object in memory.
- At this stage, it contains no image data.

loadFromFile()

- Reads pixel data from the specified file.
- Transfers the image into GPU-accessible memory.
- Returns true if loading succeeds.

Important Concept: Resource loading is typically done once during initialization, not inside the game loop.

Step 3: Loading Background (Part 2)

```
        Sprite spriteBackground;
        spriteBackground.setTexture(textureBackground);
```

This step creates a drawable object using the loaded texture.

Sprite spriteBackground;

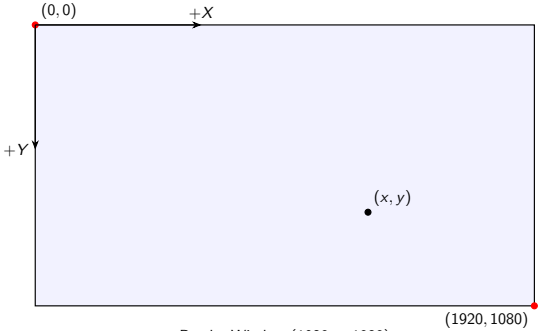
- Creates a renderable object.
- Initially has no image assigned.

setTexture(textureBackground);

- Associates the texture with the sprite.
- The sprite now references image data.
- The sprite can now be drawn to the window.

Rendering Flow:
File → Texture (Memory) → Sprite (Object) → Window

Coordinate System in SFML



Render Window (1920×1080)

- The origin $(0, 0)$ is located at the top-left corner of the window.
- The X-axis increases toward the right.
- The Y-axis increases downward (opposite to the traditional Cartesian system).
- Every sprite position is defined using this coordinate system.

Loading Player and Axe Textures

```
        Texture texturePlayer;
        texturePlayer.loadFromFile("graphics/player.png");

        Texture textureAxe;
        textureAxe.loadFromFile("graphics/axe.png");
```

Resource Initialization Phase

- Each visual object requires its own texture.
- Images are loaded from disk into memory.
- This operation should occur once during initialization.
- Loading inside the game loop would reduce performance.

Concept: File → Texture (Memory Resource)

Creating Player and Axe Sprites

```
        Sprite spritePlayer(texturePlayer);
        Sprite spriteAxe(textureAxe);
```

Sprite Construction

- Sprite is a drawable entity.
- It references the texture passed in constructor.
- Multiple sprites can share textures if required.

Important Distinction:

- Texture = Image Data
- Sprite = Scene Object

Positioning Sprites in the Scene

```
        spritePlayer.setPosition(580, 720);
        spriteAxe.setPosition(700, 830);
```

Coordinate Placement

- setPosition(x, y) defines sprite location.
- Coordinates follow SFML system:

$$(0, 0) \text{ is top-left}$$

- Values are measured in pixels.

Interpretation: Player and axe are placed near bottom of 1920×1080 window.

Rendering Pipeline: Drawing Objects

```
        window.clear();

        window.draw(spriteBackground);
        window.draw(spriteTree);
        window.draw(spritePlayer);
        window.draw(spriteAxe);

        window.display();
```

This sequence renders the entire scene for one frame.

Rendering Order and Frame Composition

1. **window.clear()**
   - Clears previous frame.
   - Prevents visual artifacts.
2. **window.draw(object)**
   - Draws objects in the order specified.
   - Earlier draw calls appear behind later ones.
   - This defines layering (background → foreground).
3. **window.display()**
   - Swaps the back buffer to the screen.
   - Completes the frame rendering cycle.

Rendering Model:
State → Draw Commands → Frame Output

Displaying Dynamic Score: Why ¡sstream¿?

```
#include <sstream>
using namespace std;
```

The score changes during gameplay, so we must convert numeric values into text.

Problem: SFML Text requires a string, not an integer.

Solution: Use stringstream to concatenate text and numbers safely.

Loading Font and Creating Score Text

```
        Font font;
        font.loadFromFile("font/KOMIKAP_.ttf");

        Text scoreText;
        scoreText.setFont(font);
        scoreText.setCharacterSize(100);
        scoreText.setFillColor(Color::Red);
        scoreText.setPosition(20, 20);
```

- Font must be loaded before creating Text objects.
- Text object defines:
  - Font style
  - Size
  - Color
  - Screen position

Text is just another drawable object.

Updating Score Dynamically

```
        int score = 0;
        stringstream ss;
        ss << "Score = " << score;
        scoreText.setString(ss.str());
```

Process:

- Insert text and integer into stream.
- Convert stream into string.
- Assign string to SFML Text object.

This allows dynamic text updates every frame.

Centering Text on Screen

```
        FloatRect textRect = messageText.getLocalBounds();

        messageText.setOrigin(textRect.left + textRect.width / 2.0f,textRect.top +
            textRect.height / 2.0f);

        messageText.setPosition(1920 / 2.0f, 1080 / 2.0f);
```

- getLocalBounds() returns text dimensions.
- setOrigin() moves reference point to center.
- setPosition() places it at screen center.

This ensures perfect alignment regardless of text length.

Designing the Time Bar

```
        RectangleShape timeBar;
        float timeBarStartWidth = 400;
        float timeBarHeight = 80;

        timeBar.setSize(Vector2f(timeBarStartWidth, timeBarHeight));
        timeBar.setFillColor(Color::Red);
        timeBar.setPosition((1920 / 2) - timeBarStartWidth / 2, 980);
```

The time bar visually represents remaining time. Width decreases as time decreases.

Delta Time and Time Reduction

```
        float timeRemaining = 6.0f;
        float timeBarWidthPerSecond =   timeBarStartWidth / timeRemaining;

        Clock clock;
        Time dt;
```

Concept:

$$timeRemaining = timeRemaining - \Delta t$$

- Clock measures elapsed time.
- dt ensures frame-rate independence.
- Time bar shrinks proportionally.

Pause and Start Logic

```
        bool paused = true;

        if (event.type == Event::KeyPressed &&event.key.code == Keyboard::Enter)
        {
            paused = false;
            clock.restart();
        }
```

- Game begins in paused state.
- Pressing Enter starts the game.
- Clock resets when game starts.

Paused state prevents time from decreasing.

Real-Time Update: Timer Evolution

```
if (!paused)
{
    dt = clock.restart();
    timeRemaining -= dt.asSeconds();

    timeBar.setSize(Vector2f(
        timeBarWidthPerSecond *timeRemaining
        ,timeBarHeight));

    if (timeRemaining <= 0.0f)
    {
        paused = true;
        messageText.setString("Out of Time!"
            );
    }
}
```

State Update Logic

- Executed only when paused = false.
- Δt ensures frame-rate independence.
- Timer decreases continuously.
- Time bar width scales linearly with time.
- When time ≤ 0:
  - Game transitions to paused state.
  - Message updates dynamically.

$$T(t+1) = T(t) - \Delta t$$

Conditional Rendering Based on Game State

```
if (paused)
    window.draw(messageText);
```

Rendering Control Logic

- Drawing depends on game state.
- If paused = true:
  - Start or Game Over message appears.
- If paused = false:
  - Message is not rendered.

Key Idea

- Game logic and rendering are both controlled by the same state variable.
- State → Conditional Update → Conditional Draw

Time Bar Reduction Analysis

The red bar width decreases linearly with time: $W(t) = \frac{W_0}{T_0} \cdot T(t)$
This creates a real-time visual feedback mechanism.

Mathematical Model of the Timer System

The timer is a continuous-time variable updated discretely at each frame iteration.

Let:

$$T(t) = \text{remaining time at frame } t$$

$$\Delta t = \text{elapsed time between frames}$$

Update Rule (when not paused):

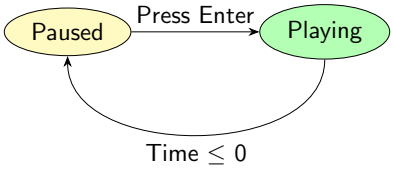
$$T(t+1) = T(t) - \Delta t$$

Game Over Condition:

$$T(t) \leq 0 \Rightarrow paused = true$$

Time bar width is proportional to remaining time:

Pause and Play: Finite State Machine



States:

- Paused — Timer does not decrease.
- Playing — Timer updates every frame.

State transitions are triggered by:

- User input (Enter key)
- Timer reaching zero

Step 4: Player Side Enumeration (Part 1)

```
        enum class Side { LEFT, RIGHT, NONE };
        Side playerSide = Side::LEFT;
```

An **enumeration (enum)** is a user-defined data type that represents a fixed set of named constants.

In our game:

- The player can only be on:
  - LEFT
  - RIGHT
  - NONE (initial state)
- No other values are logically valid.

Instead of using numbers like 0, 1, 2, we use meaningful names.

Why Not Use Integers? (Part 2)

```
int playerSide = 0;  // 0 = LEFT ?
```

Using integers creates ambiguity and risk.

- What does 0 mean?
- What if someone assigns:

```
playerSide = 100;
```

- The compiler will not stop it.
- Invalid states become possible.

Problem: Integers do not restrict values to valid logical states.

Why enum class is Safer (Part 3)

```
enum class Side { LEFT, RIGHT, NONE };
Side playerSide = Side::LEFT;
```

Advantages of enum class:

- Strongly typed (cannot assign random integers).
- Scoped (must write Side::LEFT).
- Prevents naming conflicts.
- Improves readability.

Invalid assignment:

```
playerSide = 1;    // Compilation error
```

This ensures the game state remains logically consistent.

Using enum in Branch Collision Logic

```
        enum class Side { LEFT, RIGHT, NONE };
        Side playerSide = Side::LEFT;
        if (branchPositions[5] == playerSide)
        {
            // Player dies
            alive = false;
        }
```

Why enum is powerful here:

- Both branchPositions[] and playerSide use the same type (Side).
- Compiler ensures type consistency.
- Comparison is meaningful and readable:

$$B_{N-1}(t) = P(t)$$

- No ambiguous integer comparison.

This models collision as a comparison of two finite states.

enum vs enum class

Traditional enum

```
enum Side { LEFT, RIGHT };
```

- Values implicitly convert to int.
- Names enter global scope.
- Possible naming conflicts.

Modern enum class (C++11)

```
enum class Side { LEFT, RIGHT };
```

- Strongly typed.
- No implicit conversion to int.
- Scoped usage: Side::LEFT.
- Safer for game state modeling.

For large systems and games, enum class is preferred.

Memory Representation of enum

Although enum represents symbolic names, internally it is stored as an integer.
Example:

$$LEFT = 0, \quad RIGHT = 1, \quad NONE = 2$$

Memory representation:

- Typically stored as an int (4 bytes).
- Can be explicitly specified:

$$enum\ class\ Side : char\{LEFT, RIGHT, NONE\};$$

- Then it occupies 1 byte.

Important Insight: Enum improves readability and safety, but runtime comparison is still integer comparison.

Branch System Logic: Discrete State Transition

The falling branch effect in Timber Man is not physics-based. It is implemented using a discrete-time state transition on an array structure.

Let:

$$B_i(t) = \text{branch state at index } i \text{ at time } t$$

where:

- $i \in \{0, 1, \ldots, N-1\}$
- $i = 0$ represents the top of the tree
- $i = N-1$ represents the lowest visible branch

Branch System Logic: Discrete State Transition

State Transition Rule:

$$B_i(t+1) = B_{i-1}(t), \quad i = 1, 2, \ldots, N-1$$

This means:

- Each branch shifts one position downward.
- The previous state is copied to the next index.
- The entire system evolves in discrete time steps.

Key Insight: The visual illusion of falling branches is achieved through array index shifting, not through gravity simulation.

Branch System Logic: Discrete State Transition

State transition from t to $t+1$.

Step 5: Branch Array

```
        const int NUM_BRANCHES = 6;

        Sprite branches[NUM_BRANCHES];
        Side branchPositions[NUM_BRANCHES];
```

Step 6: Updating Branches

```
void updateBranches(int seed)
{
    for(int j = NUM_BRANCHES - 1; j > 0; j--)
    {
        branchPositions[j] = branchPositions[j - 1];
    }

    if(seed == 0)
    branchPositions[0] = Side::LEFT;
    else
    branchPositions[0] = Side::RIGHT;
}
```

Collision Detection

Player dies if:

$$branchPositions[5] == playerSide$$

Only last branch matters because it reaches player level.

Step 7: Timer Using Clock

```
        Clock clock;
        Time dt = clock.restart();

        timeRemaining -= dt.asSeconds();
```

Delta time ensures frame-rate independence.

Scoring Mechanism

- Each chop increases score.
- Each chop increases time slightly.
- Difficulty increases gradually.

Game Over Logic

Game ends if:

- Time ≤ 0
- Branch collision occurs

Complete Game Flow

1. Initialize Window and Textures
2. Create Sprites
3. Initialize Branch Array
4. Start Game Loop
5. Handle Input
6. Update Branches
7. Check Collision
8. Update Score and Time
9. Draw Everything

Questions ?

Thank You!